



Snake Game

Using Assembly Language
(emu8086)

Mariam Bakr Mohamed 2024/06881

Carol Emil Ishak 2024/04795

Rahma Meselhy Ahmed 2024/0851

Malak Ahmed Hendawy 2024/03803

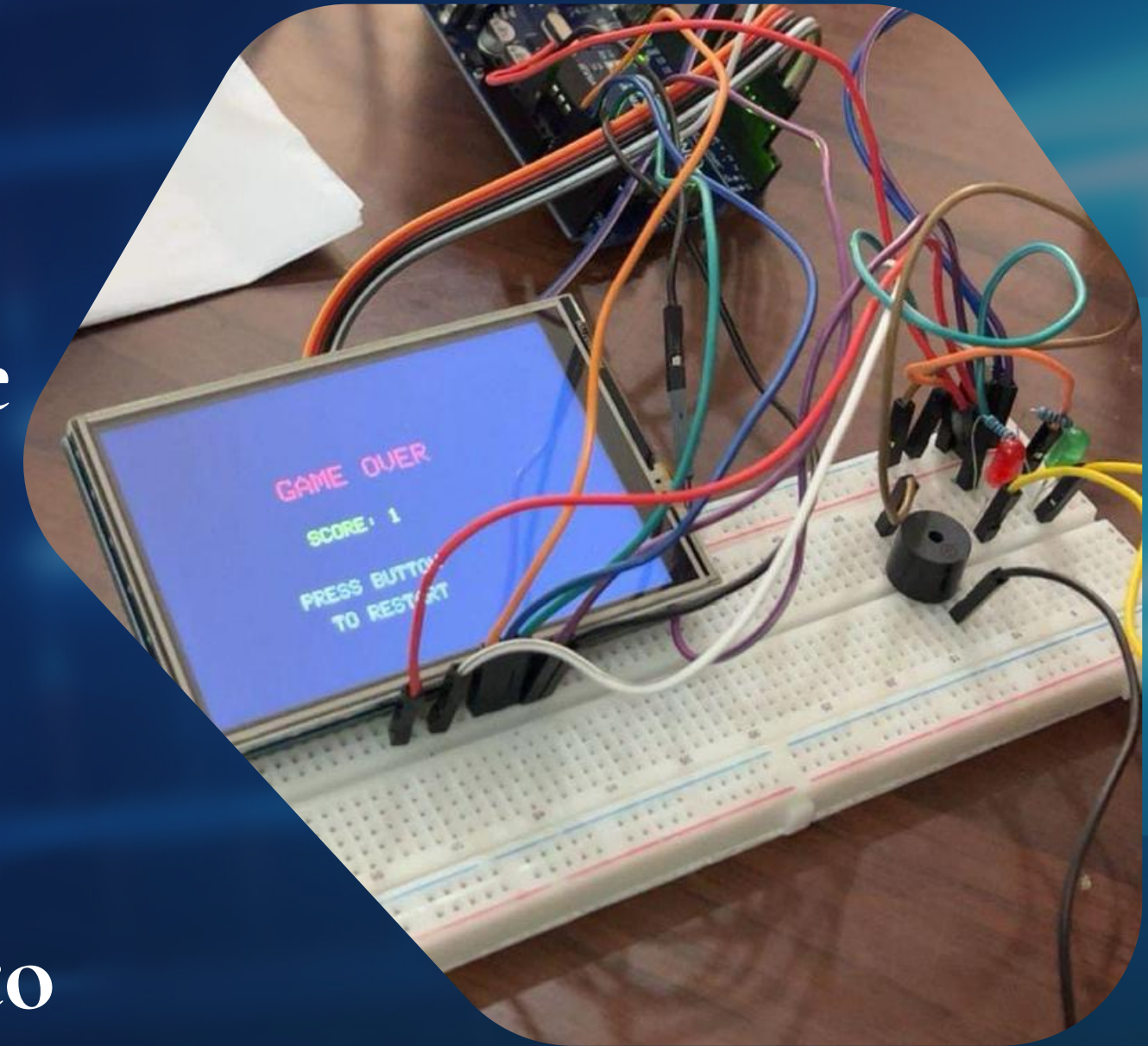
Farah Yousri Nasr 2024/04366

Merna Nagy Gouda 2024/07377

Project Idea

The project is a classic Snake Game implemented using Assembly Language (emu8086) to demonstrate low-level hardware control and computer organization concepts.

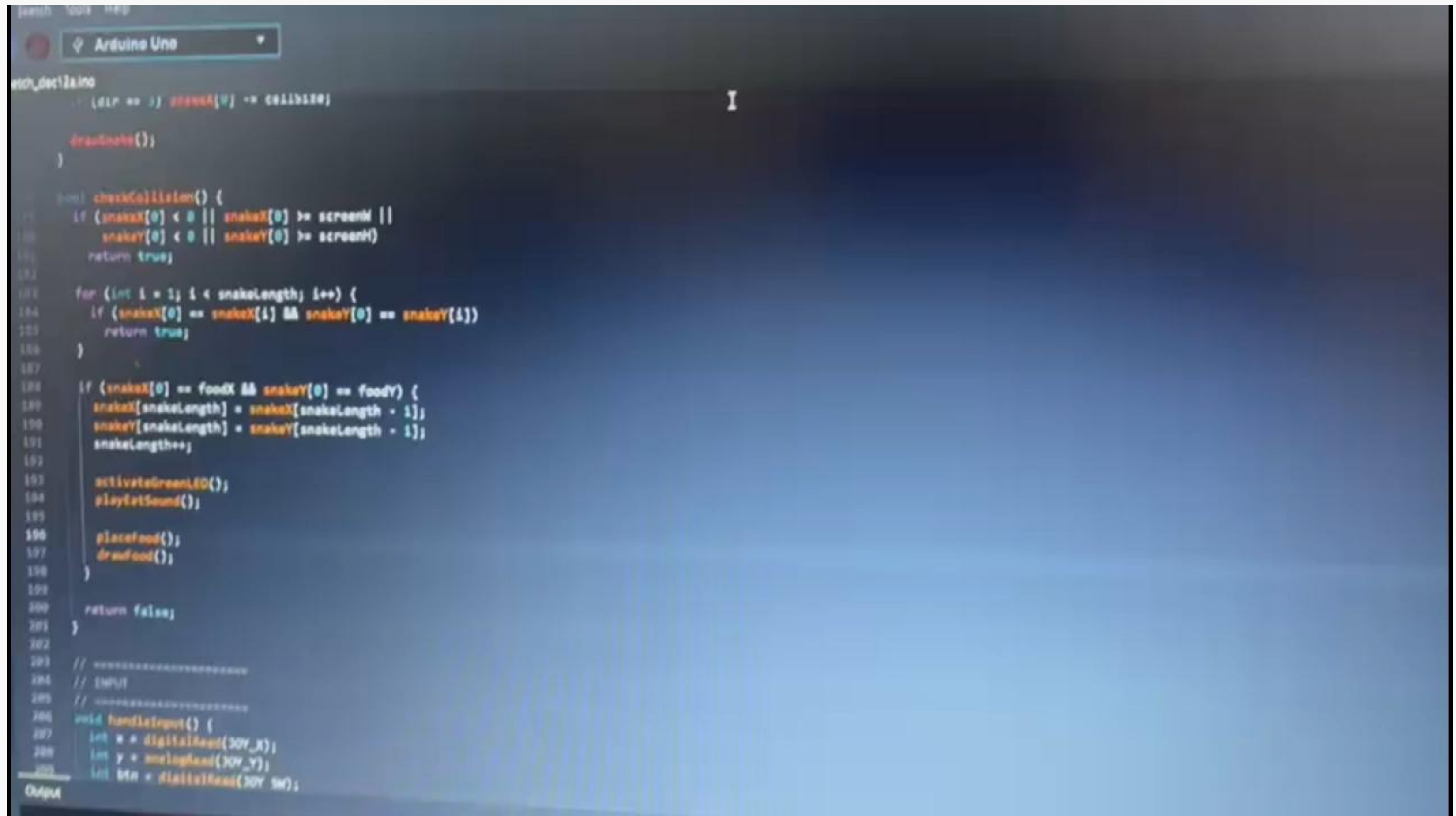
In addition to the software implementation, the game was also implemented on real hardware using Arduino Uno and TFT-LCD to show how software interacts with hardware components.



Implementation Steps

- *Write the Snake Game logic using 8086 Assembly Language on emu8086.*
- *Implement snake movement, food generation, and collision detection.*
- *Display the game on the emulator screen.*
- *Build the hardware circuit using:*
 - *Arduino Uno*
 - *TFT-LCD display*
 - *Breadboard and connecting wires*
 - *Program the Arduino to display and control the Snake Game on the LCD.*

Demonstration video



```
search 100% Help
Arduino Uno
snake_game.ino
1: (dir == 0) snakeX[0] -= collision;
2: drawSnake();
3: }
4:
5: bool checkCollision() {
6:   if (snakeX[0] < 0 || snakeX[0] >= screenW ||
7:       snakeY[0] < 0 || snakeY[0] >= screenH)
8:     return true;
9:
10:   for (int i = 1; i < snakeLength; i++) {
11:     if (snakeX[0] == snakeX[i] && snakeY[0] == snakeY[i])
12:       return true;
13:   }
14:
15:   if (snakeX[0] == foodX && snakeY[0] == foodY) {
16:     snakeX[snakeLength] = snakeX[snakeLength - 1];
17:     snakeY[snakeLength] = snakeY[snakeLength - 1];
18:     snakeLength++;
19:
20:     activateGreenLED();
21:     playEatSound();
22:
23:     placeFood();
24:     drawFood();
25:   }
26:
27:   return false;
28: }
29:
30: // =====
31: // INPUT
32: // =====
33: void handleInput() {
34:   int x = digitalRead(XY_X);
35:   int y = analogRead(XY_Y);
36:   int btn = digitalRead(XY_BTN);
37: }
```

Output

Assembly vs Arduino – Snake Speed Difference

Arduino provides faster and smoother snake movement due to its real-time microcontroller execution, while Assembly (emu8086) relies on software delay loops in a simulated environment, resulting in slower and less flexible speed control.